**Luiss**
Libera Università Internazionale
degli Studi Sociali Guido Carli

# Lab on Block Cipher Operation Modes

## Data Privacy and Security

**Data Science and Management (DASMA)**

LUISS

# From Perfect Secrecy to Practical Security

> OTP is **theoretically perfect**, but **impractical** in real systems and **dangerous** when misused.

# From Perfect Secrecy to Practical Security

> OTP is **theoretically perfect**, but **impractical** in real systems and **dangerous** when misused.

OTP limitations:

- Key must be **as long as the message** $\Longrightarrow$ leads to storage and distribution problems.
- Key must **never be reused** $\Longrightarrow$ hard to generate many keys at scale.
- Key must be **shared securely** in advance $\Longrightarrow$ requires a secure channel before communication.

# From Perfect Secrecy to Practical Security

OTP is **theoretically perfect**, but **impractical** in real systems and **dangerous** when misused.

OTP limitations:

- Key must be **as long as the message** $\Longrightarrow$ leads to storage and distribution problems.
- Key must **never be reused** $\Longrightarrow$ hard to generate many keys at scale.
- Key must be **shared securely** in advance $\Longrightarrow$ requires a secure channel before communication.

Modern symmetric ciphers replace perfect secrecy with **computational security**, enabling short, reusable keys.

- **Perfect secrecy**: the ciphertext reveals **no information** about the plaintext, even to an all-powerful attacker.
- **Computational security**: the ciphertext could theoretically reveal information about the plaintext, but extracting it is **computationally infeasible** with realistic computing power.

# Symmetric Cipher Approaches

Two main approaches:

# Symmetric Cipher Approaches

Two main approaches:

1. **Stream** ciphers:
   - Inspired by OTP.
   - Given a secret key, generate a byte of stream called *keystream* that has the same length as the message.
   - Encrypt/decrypt bits from x individually using a XOR operation with the keystream (as in OTP).

# Symmetric Cipher Approaches

Two main approaches:

1. **Stream** ciphers:
   - Inspired by OTP.
   - Given a secret key, generate a byte of stream called *keystream* that has the same length as the message.
   - Encrypt/decrypt bits from x individually using a XOR operation with the keystream (as in OTP).

2. **Block** ciphers:
   - Split the message x in blocks of fixed size.
   - Encrypt/decrypt each block, then combine the results according to the operation mode.
   - Different *operation mode*: process blocks independently, chain blocks, etc.

# Symmetric Cipher Approaches

Two main approaches:

1. **Stream** ciphers:
   - Inspired by OTP.
   - Given a secret key, generate a byte of stream called *keystream* that has the same length as the message.
   - Encrypt/decrypt bits from x individually using a XOR operation with the keystream (as in OTP).

2. **Block** ciphers:
   - Split the message x in blocks of fixed size.
   - Encrypt/decrypt each block, then combine the results according to the operation mode.
   - Different *operation mode*: process blocks independently, chain blocks, etc.

> Modern symmetric encryption algorithms (e.g., **AES**, DES, ...) are built on these two primitives.

# Block Ciphers

# Block Ciphers

Given:

- A block $x_i$ of the message $x$ of $h$ bits ($h$ is <u>fixed</u> by the block cipher).
- A key $k$ of $n$ bits ($n$ <u>fixed</u> by the block cipher).
- $n$ can be different from $h$.

Then:

$$x_i \longrightarrow \boxed{E_k(x_i)} \longrightarrow y_i$$

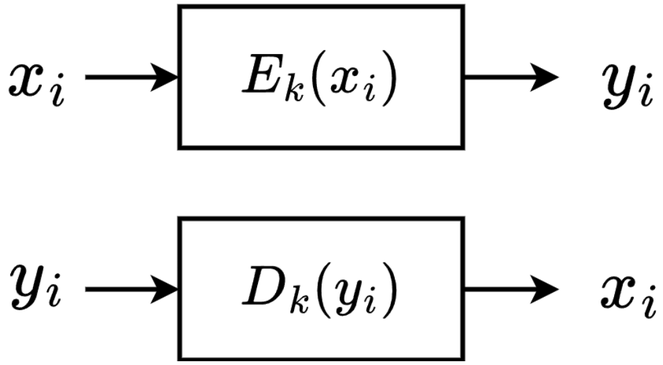
$$y_i \longrightarrow \boxed{D_k(y_i)} \longrightarrow x_i$$

# Block Ciphers

Given:

- A block $x_i$ of the message $x$ of $h$ bits ($h$ is <u>fixed</u> by the block cipher).
- A key $k$ of $n$ bits ($n$ <u>fixed</u> by the block cipher).
- $n$ can be different from $h$.

Then:




The full plaintext/ciphertext is obtained by combining the processed blocks according to the **mode of operation**.

# What if the message is larger than the block size?

Strategy:

1. **Divide** the message into blocks of size $h$.
2. Use a **mode of operation** to combine the blocks during encryption.

Common modes of operation:

- Electronic Code Block (**ECB**)
- Cipher Block Chaining (**CBC**)
- Counter Mode (**CTR**)
- ...

# What if the message is larger than the block size?

Strategy:

1. **Divide** the message into blocks of size $h$.
2. Use a **mode of operation** to combine the blocks during encryption.

Common modes of operation:

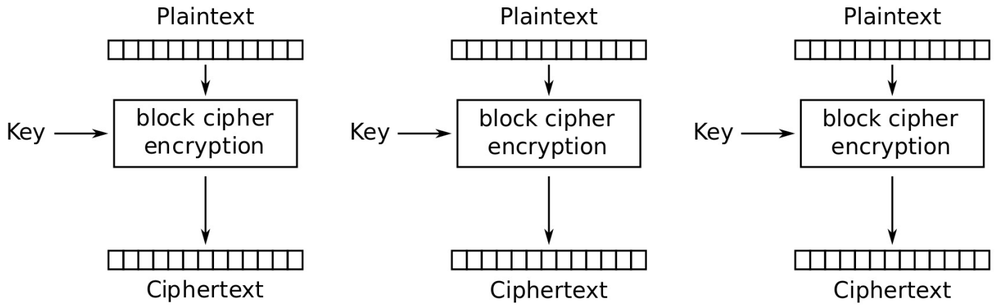
- Electronic Code Block (**ECB**)
- Cipher Block Chaining (**CBC**)
- Counter Mode (**CTR**)
- ...

> Choosing the right mode affects both security and how blocks are processed.

# Electronic Code Block (ECB)

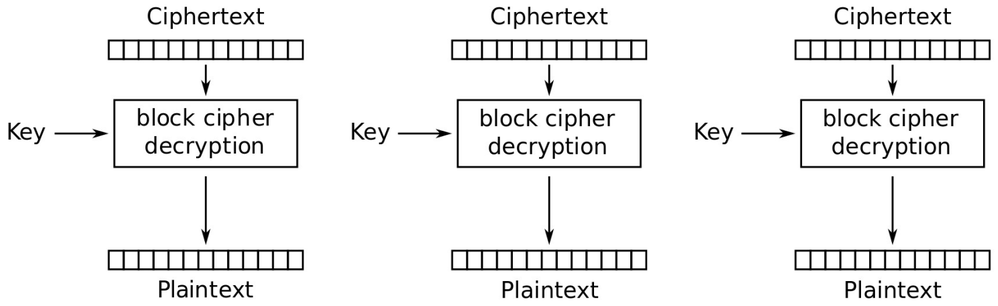
**IDEA**: Encrypt each plaintext block independently, then append them to form the full ciphertext.

Encryption:




Electronic Codebook (ECB) mode encryption

Decryption:




Electronic Codebook (ECB) mode decryption

Notice that:

- $Enc_k(M) = Enc_k(M_1) \parallel Enc_k(M_2) \parallel \ldots \parallel Enc_k(M_n)$
- $Dec_k(C) = Dec_k(C_1) \parallel Dec_k(C_2) \parallel \ldots \parallel Dec_k(C_n)$
- Each block is **treated separately** i.e. there's **no chaining or linking** between blocks.

# Electronic Code Block (ECB) - Weakness

Problem:

- Identical plaintext blocks produce identical ciphertext blocks.
- Patterns in the plaintext can **leak through the ciphertext**.

# Electronic Code Block (ECB) - Weakness

Problem:

- Identical plaintext blocks produce identical ciphertext blocks.
- Patterns in the plaintext can **leak through the ciphertext**.

```
def ECB_ENC(key, msg):
    blocks = [msg[i:i+16] for i in range(0, len(msg), 16)]
    return b''.join(ENC(key, block) for block in blocks)

K = b'YELLOW SUBMARINE'
M1 = b'16 bytes message'
M2 = b' another 16-b msg'

print(ECB_ENC(K, M1).hex())
#3c19892b47fa369d7301dd1928eb06ca
print(ECB_ENC(K, M2).hex())
#9d28a90864f1f26a7feb218af64dd51a
print(ECB_ENC(K, M1 + M2).hex())
#3c19892b47fa369d7301dd1928eb06ca9d28a90864f1f26a7feb218af64dd51a
```

# Electronic Code Block (ECB) - Exploitation

- **Blocks reordering**: the message "Bob transfers $10 to Luca" could be manipulated into "Luca transfers $10 to Bob".
- **Blocks can be replaced, removed, appended**: the message "Transfer $10 to Oscar" could be manipulated into "Transfer $10000 to Oscar".
- **Encrypting the same message twice will generate the same ciphertext**: anyone can detect you are sending the same message twice, which could be a privacy issue!

# Electronic Code Block (ECB) - Exploitation

- **Blocks reordering**: the message "Bob transfers $10 to Luca" could be manipulated into "Luca transfers $10 to Bob".
- **Blocks can be replaced, removed, appended**: the message "Transfer $10 to Oscar" could be manipulated into "Transfer $10000 to Oscar".
- **Encrypting the same message twice will generate the same ciphertext**: anyone can detect you are sending the same message twice, which could be a privacy issue!

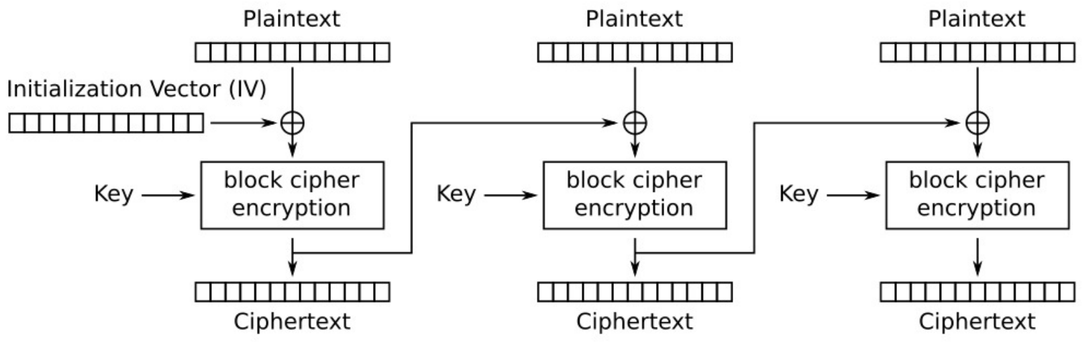
Example: assume a bank stores account infos like so:

```
#m1 = [USER:RALLI][CREDIT:10€]
c1.hex() = [8f3a91c4e2d6b7a09c5f1e8a42bd7c11][4d9b6e2a7f31c8e5a04b2d6f91aa73ce]

#m2 = [USER:COPPA][CREDIT:1000€]
c2.hex() = [b71c4f2a9e803d6c5a18e29f04db7c92][a3f9c2d8715e04b6cc9a21f8e73d0b44]

#Swap CREDIT block from m2 with CREDIT block from m1

#[USER:RALLI][CREDIT:1000€]
[8f3a91c4e2d6b7a09c5f1e8a42bd7c11][a3f9c2d8715e04b6cc9a21f8e73d0b44]   #jackpot!
```

# Cipher Block Chaining (CBC)

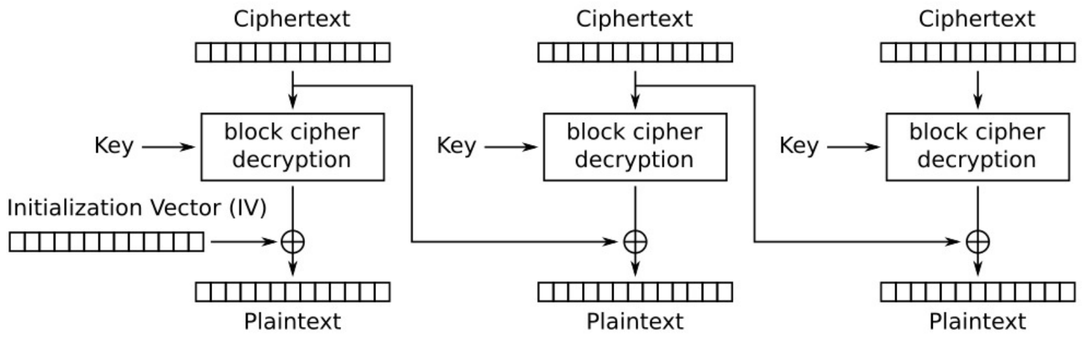
**IDEA**: Each plaintext block is XORed with the previous ciphertext block.

Encryption:




Cipher Block Chaining (CBC) mode encryption

Decryption:




Cipher Block Chaining (CBC) mode decryption

Notice that:

- $Enc_k(M) = Enc_k(M_1 \oplus IV) \parallel Enc_k(M_2 \oplus C_1) \parallel \ldots \parallel Enc_k(M_n \oplus C_{n-1})$
- $Dec_k(C) = Dec_k(C_1) \oplus IV \parallel Dec_k(C_2) \oplus C_1 \parallel \ldots \parallel Dec_k(C_n) \oplus C_{n-1}$
- A **random IV initializes the chaining**, causing each block to influence the next.

# Cipher Block Chaining (CBC) - Weaknesses

**Bit flipping**: flipping (changing) a bit in a ciphertext block...

- completely corrupts the corresponding plaintext block;
- flips the same bit in the next plaintext block.

This allows **controlled modifications** of the decrypted message (**malleability**).

# Cipher Block Chaining (CBC) - Weaknesses

**Bit flipping**: flipping (changing) a bit in a ciphertext block...

- completely corrupts the corresponding plaintext block;
- flips the same bit in the next plaintext block.

This allows **controlled modifications** of the decrypted message (**malleability**).

Initialization Vector (IV) requirements:

- It must be **random**, and **never reused** with the same key.
- Reusing it can lead to information leakage about the first plaintext block.
- It must be **authenticated**, otherwise attackers can modify it to control the first plaintext block.

# Cipher Block Chaining (CBC) - Exploitation

Assumptions:

- The **IV** is **prepended to the ciphertext** and sent to the user: $ciphertext = IV \parallel C_1 \parallel C_2 \parallel \ldots$
- CBC is used **without authentication**.

# Cipher Block Chaining (CBC) - Exploitation

Assumptions:

- The **IV** is **prepended to the ciphertext** and sent to the user: $ciphertext = IV \parallel C_1 \parallel C_2 \parallel \ldots$
- CBC is used **without authentication**.

Observations:

- First step in CBC's decryption algorithm: $Dec_k(C) = (Dec_k(C_1) \oplus IV) = M_1$
- The attacker has control over the IV => by flipping selected bits of the IV, they can **predictably modify the first plaintext block**.

# Cipher Block Chaining (CBC) - Exploitation

Assumptions:

- The **IV** is **prepended to the ciphertext** and sent to the user: $ciphertext = IV \parallel C_1 \parallel C_2 \parallel \ldots$
- CBC is used **without authentication**.

Observations:

- First step in CBC's decryption algorithm: $Dec_k(C) = (Dec_k(C_1) \oplus IV) = M_1$
- The attacker has control over the IV => by flipping selected bits of the IV, they can **predictably modify the first plaintext block**.

**Example**: suppose the attacker wants to change the first bit $m$ of plaintext message $M$ into a desired bit $m'$...

$$IV' = IV \oplus m \oplus m'$$

$$Dec_k(C) = (Dec_k(C_1) \oplus IV') \mid \cdots = (Dec_k(C_1) \oplus IV \oplus m \oplus m') \mid \ldots$$

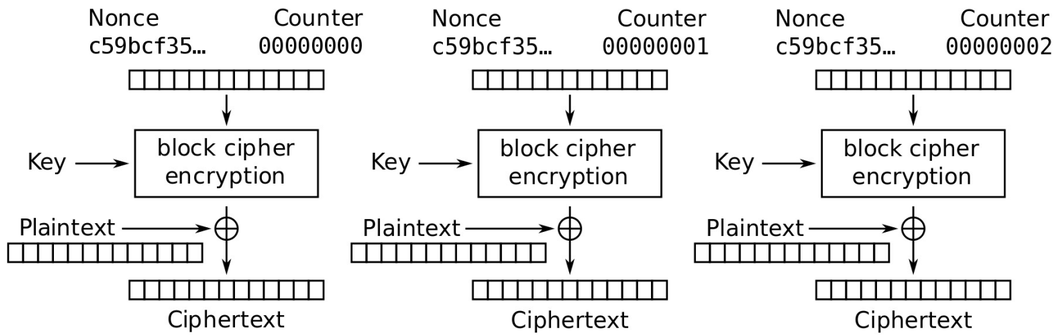
$$= (M_1 \oplus m \oplus m') \mid \cdots = (M') \mid \ldots$$

Result: the decrypted message $M'$ is identical to $M$, **except for the first bit** which is changed from $m$ to $m'$.

# Counter Mode (CTR)

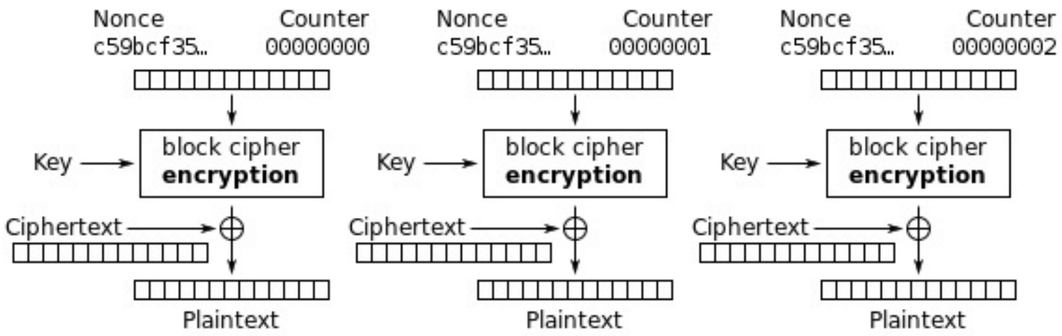
**IDEA**: Like OTP, but each block uses a different key derived from nonce + counter.

Encryption:




Counter (CTR) mode encryption

Decryption:




Counter (CTR) mode decryption

Notice that:

- $Enc_k(M) = (M_1 \oplus Enc_k(\text{nonce}|0)) \mid (M_2 \oplus Enc_k(\text{nonce}|1)) \mid \ldots \mid (M_n \oplus Enc_k(\text{nonce}|n))$
- $Dec_k(C) = (C_1 \oplus Enc_k(\text{nonce}|0)) \mid (C_2 \oplus Enc_k(\text{nonce}|1)) \mid \ldots \mid (C_n \oplus Enc_k(\text{nonce}|n))$
- nonce + counter generate a **keystream**, which is then **XORed with each block independently**.

# Counter Mode (CTR) - Weaknesses

Nonce requirements:

- Must be **unique** for every encryption under the same key.
- Reusing a nonce causes **keystream reuse**, leaking relationships between plaintexts.

# Counter Mode (CTR) - Weaknesses

Nonce requirements:

- Must be **unique** for every encryption under the same key.
- Reusing a nonce causes **keystream reuse**, leaking relationships between plaintexts.

Counter requirements:

- Must **change for every block**.
- Must **never repeat** within the same encryption.
- A static or repeating counter breaks keystream uniqueness.

# Counter Mode (CTR) - Weaknesses

Nonce requirements:

- Must be **unique** for every encryption under the same key.
- Reusing a nonce causes **keystream reuse**, leaking relationships between plaintexts.

Counter requirements:

- Must **change for every block**.
- Must **never repeat** within the same encryption.
- A static or repeating counter breaks keystream uniqueness.

> If both nonce and counter are static, CTR **degenerates into OTP with key reuse**.

# Counter Mode (CTR) - Exploitation

**Malleability** (Bit-Flipping)

- Each step in CTR decryption is $M_i = C_i \oplus Enc_k(\text{nonce}|i)$
- Attacker can flip a bit in the ciphertext: $C'_i = C_i \oplus \Delta \quad \text{where} \quad \Delta = m \oplus m'$
- Decryption: $M'_i = C'_i \oplus Enc_k(\text{nonce}|i) = C_i \oplus \Delta \oplus Enc_k(\text{nonce}|i) = M_i \oplus \Delta$

Result: the decrypted message $M'$ is identical to $M$ except for the **bit $m$ being changed into $m'$**.

# Counter Mode (CTR) - Exploitation

**Malleability** (Bit-Flipping)

- Each step in CTR decryption is $M_i = C_i \oplus Enc_k(\text{nonce}|i)$
- Attacker can flip a bit in the ciphertext: $C_i' = C_i \oplus \Delta \quad \text{where} \quad \Delta = m \oplus m'$
- Decryption: $M_i' = C_i' \oplus Enc_k(\text{nonce}|i) = C_i \oplus \Delta \oplus Enc_k(\text{nonce}|i) = M_i \oplus \Delta$

Result: the decrypted message $M'$ is identical to $M$ except for the **bit $m$ being changed into $m'$**.

**Nonce Reuse**

- Given two messages encrypted with the same nonce:

$$C_i = M_i \oplus Enc_k(\text{nonce}|i)$$

$$C_i' = M_i' \oplus Enc_k(\text{nonce}|i)$$

- Attacker can compute: $C_i \oplus C_i' = M_i \oplus Enc_k(\text{nonce}|i) \oplus M_i' \oplus Enc_k(\text{nonce}|i) = M_i \oplus M_i'$

Result: the attacker can use **cribbing/frequency analysis** to reconstruct the plaintext.

# Advanced Encryption Standard (AES)

> The most widely used symmetric encryption algorithm in modern cryptography,

Characteristics:

- Block size: **16 bytes**.
- Key lengths: 128, 192, or 256 bits.

It implements modes of operations:

- MODE_ECB
- MODE_CBC
- MODE_CTR
- ...

# AES in Python

Installation: `pip install pycryptodome`

```
from Crypto.Cipher import AES

cipher = AES.new(KEY, AES.MODE_CBC, IV)
cipher.encrypt(data)
```

If the data length isn't a multiple of 16 bytes, **it must be padded**.

```
from Crypto.Util.Padding import pad, unpad

padded_data = pad(data, 16)
cipher.encrypt(padded_data)
```

# AES in Python

AES **ECB_MODE** implementation:

```
def encrypt_ecb(plaintext):
    cipher = AES.new(key, AES.MODE_ECB)
    ciphertext = cipher.encrypt(pad(data, AES.block_size))
    return ciphertext.hex()
```

AES **CBC_MODE** implementation:

```
def encrypt_cbc(plaintext):
    iv = get_random_bytes(16)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(data, AES.block_size))
    return iv, ciphertext.hex()
```

AES **CTR_MODE** implementation

```
def encrypt_ctr(plaintext):
    nonce = get_random_bytes(8)
    cipher = AES.new(KEY, AES.MODE_CTR, nonce=nonce)
    ciphertext = cipher.encrypt(plaintext.encode())
    return ciphertext.hex()
```

# Time to Cook

# Entropy Vault II

Crypto Challenge 04

# Entropy Vault II - Reconnaissance

EntropyVault2 (500 points) crypto

New cipher, old habits.

**https://crypto04.webhack.it**

Flag

Submit

What can we observe? The challenge...

- claims to use **AES-CTR** encryption;
- provides an **encryption oracle** for user-chosen messages;
- allows us to **retrieve the encrypted flag**.

# Entropy Vault II - Hypothesis

```python
#fixed nonce (set globally once)
NONCE = os.getenv("NONCE", "noncenon").encode()

def encrypt(plaintext):
    try:
        cipher = AES.new(KEY, AES.MODE_CTR, nonce=NONCE)
        ciphertext = cipher.encrypt(plaintext.encode())
        return ciphertext.hex()
    except Exception as e:
        print(f"Encryption error: {e}")
        return None
```

# Entropy Vault II - Hypothesis

```python
#fixed nonce (set globally once)
NONCE = os.getenv("NONCE", "noncenon").encode()

def encrypt(plaintext):
    try:
        cipher = AES.new(KEY, AES.MODE_CTR, nonce=NONCE)
        ciphertext = cipher.encrypt(plaintext.encode())
        return ciphertext.hex()
    except Exception as e:
        print(f"Encryption error: {e}")
        return None
```

Nonce reuse in AES-CTR causes keystream reuse, effectively **turning it into a OTP with key reuse**.

# Entropy Vault II - Hypothesis

```python
#fixed nonce (set globally once)
NONCE = os.getenv("NONCE", "noncenon").encode()

def encrypt(plaintext):
    try:
        cipher = AES.new(KEY, AES.MODE_CTR, nonce=NONCE)
        ciphertext = cipher.encrypt(plaintext.encode())
        return ciphertext.hex()
    except Exception as e:
        print(f"Encryption error: {e}")
        return None
```

Nonce reuse in AES-CTR causes keystream reuse, effectively **turning it into a OTP with key reuse**.

We can encrypt an **arbitrary number** of messages.

# Entropy Vault II - Exploitation

1. **Encrypt a known plaintext** by submitting a message to the encryption oracle.

2. **Extract the key** by XOR-ing the known plaintext with the recovered ciphertext:

```
ciphertext ⊕ plaintext = key
```

3. **Decrypt the flag** by XOR-ing the encrypted flag with the recovered key:

```
encrypted_flag ⊕ key = flag
```

# Entropy Vault II - Exploitation

1. **Encrypt a known plaintext** by submitting a message to the encryption oracle.

2. **Extract the key** by XOR-ing the known plaintext with the recovered ciphertext:

```
ciphertext ⊕ plaintext = key
```

3. **Decrypt the flag** by XOR-ing the encrypted flag with the recovered key:

```
encrypted_flag ⊕ key = flag
```

**NOTE**: the plaintext must be **long enough** to generate a key that covers the full encrypted flag.

# Entropy Vault II - Solution Template

```python
from pwn import xor

m1 = b'...'    #put your message here
c1 = "..."     #put the encrypted message returned by the website here
c1_bytes = bytes.fromhex(c1)

c_f = "..."    #put the encrypted flag here
c_f_bytes = bytes.fromhex(c_f)

k = xor(m1,c1_bytes)
f = xor(k, c_f_bytes)

print(f.decode())
```

# ECBuy

## Crypto Challenge 05

# ECBuy - Reconnaissance

ECBuy (500 points) crypto

Blocks don't lie. But they do move.

**https://crypto05.webhack.it**

Flag

Submit

What can we observe?

- Registration grants a **$10 voucher**.
- There's a shop section that includes the flag.
- Each shop item has an **associated verification token** (displayed in the **view details** pop-up).

# ECBuy - Hypothesis

The voucher consists of **two padded 16-byte blocks** in the format [USER:username][CREDIT:amount], encrypted with **AES-ECB**.

```
def create_voucher(username, amount):
    #USER:name____ CREDIT:amount___
    user_block = f"USER:{username}".ljust(16, '_')
    credit_block = f"CREDIT:{amount}".ljust(16, '_')
    return aes_ecb_encrypt(user_block + credit_block)
```

# ECBuy - Hypothesis

The voucher consists of **two padded 16-byte blocks** in the format [USER:username][CREDIT:amount], encrypted with **AES-ECB**.

```
def create_voucher(username, amount):
    #USER:name____ CREDIT:amount___
    user_block = f"USER:{username}".ljust(16, '_')
    credit_block = f"CREDIT:{amount}".ljust(16, '_')
    return aes_ecb_encrypt(user_block + credit_block)
```

The verification token consists of **two padded 16-byte blocks** in the format [ITEM:name][CREDIT:amount], encrypted with AES-ECB.

```
def create_item_token(item_name, price):
    clean_name = ''.join(c for c in item_name if ord(c) < 128).strip()[:10]
    #ITEM:name______ CREDIT:price____
    item_block = f"ITEM:{clean_name}".ljust(16, '_')
    credit_block = f"CREDIT:{price}".ljust(16, '_')
    return aes_ecb_encrypt(item_block + credit_block)
```

# ECBuy - Hypothesis

The voucher consists of **two padded 16-byte blocks** in the format [USER:username][CREDIT:amount], encrypted with **AES-ECB**.

```
def create_voucher(username, amount):
    #USER:name____ CREDIT:amount___
    user_block = f"USER:{username}".ljust(16, '_')
    credit_block = f"CREDIT:{amount}".ljust(16, '_')
    return aes_ecb_encrypt(user_block + credit_block)
```

The verification token consists of **two padded 16-byte blocks** in the format [ITEM:name][CREDIT:amount], encrypted with AES-ECB.

```
def create_item_token(item_name, price):
    clean_name = ''.join(c for c in item_name if ord(c) < 128).strip()[:10]
    #ITEM:name______ CREDIT:price____
    item_block = f"ITEM:{clean_name}".ljust(16, '_')
    credit_block = f"CREDIT:{price}".ljust(16, '_')
    return aes_ecb_encrypt(item_block + credit_block)
```

Both voucher and verification token **share a 16-bytes block** that determines the amount.

# ECBuy - Exploitation

1. **Register** with any username.

2. **Retrieve** both the generated voucher (without using it) and the verification token of the flag item.

3. **Swap** the last block of the voucher with the last block of the verification token.

```
#Example: username 'lralli'

#Voucher: [USER:lralli______][CREDIT:10_______]
#Verification token: [ITEM:flag________][CREDIT:10000____]

#Swap: [USER:lralli______][CREDIT:10000____]
```

4. **Redeem** the new token and buy the flag.

# ECBuy - Solution Template

```
token = "..."   #put here the voucher
user_token = token[:32]
voucher_credit_token = token[32:]

flag_token = "..."   #put here the flag item's verification token
item_token = flag_token[:32]
flag_credit_token = flag_token[32:]

print("New Token: " + user_token + flag_credit_token)
```

# ChainReaction

Crypto Challenge 06

# ChainReaction - Reconnaissance

ChainReaction (500 points) crypto

One flip to ruin them all.

**https://crypto06.webhack.it**

Flag

Submit

What can we observe?

- The NFT shop lists items, each with an **associated owner**.
- Registering a user returns an **encrypted login token**.
- Login requires providing a valid token to authenticate.

# ChainReaction - Hypothesis

```
#fixed IV (set globally once)
IV = os.urandom(16)

...

def encrypt_token(username):
    padded_username = pad(username.encode(), AES.block_size)
    cipher = AES.new(KEY, AES.MODE_CBC, IV)
    ciphertext = cipher.encrypt(padded_username)
    return (IV + ciphertext).hex()
```

# ChainReaction - Hypothesis

```
#fixed IV (set globally once)
IV = os.urandom(16)

...

def encrypt_token(username):
    padded_username = pad(username.encode(), AES.block_size)
    cipher = AES.new(KEY, AES.MODE_CBC, IV)
    ciphertext = cipher.encrypt(padded_username)
    return (IV + ciphertext).hex()
```

> The fixed IV makes the produced token **malleable**, enabling **bit-flip attacks**.

# ChainReaction - Exploitation

1. **Register** with username xralli, where x can be any character but 'l' (as lralli is already registered).

2. **Retrieve** the IV from the produced login token

```
IV = token[:BLOCK_SIZE]  #as token = IV + ciphertext
```

3. **Bit-flip** the first byte of the IV with the $\Delta$ needed to login as lralli:

```
new_token = old_token ⊕ ord('x') ⊕ ord('l')

#x = char picked in step 1
```

4. **Login** with the forged token and view the legendary NFT details to obtain the flag.

# ChainReaction - Solution Template

```
username1 = b"xralli"
admin = b"lralli"

token = "..."   #put here the login token for xralli

token = bytearray(bytes.fromhex(token))

token[0] ^= username1[0] ^ admin[0]

print(token.hex())
```